

GridPool V2.1

Ranked Proof Accounting, Payout Variance, and Bitcoin-Boundary Consensus

GridPool research packet - revised technical handout

2026-07-17

Contents

1	Executive Summary	2
2	What Is New In This Revision	2
3	The Ranked-Proof Idea	3
4	V2.1 State and Payout Construction	4
4.1	Ordinary Bitcoin block	5
4.2	GridPool block	5
5	Boundary Races and Merge-Forward Recovery	5
6	Findings	6
6.1	Actual BTC payout variance	6
6.2	Pool hopping	8
6.3	Block withholding	9
6.4	Selective inclusion and active-state anchoring	9
6.5	Immediate boundary disagreement by relay profile	10
7	Preliminary Live Relay Telemetry	10
7.1	Pulse-proof arrivals	10
8	Security Claims and Limits	11
9	Regression and Implementation Audit	11
10	What The Evidence Supports	12
11	Immediate Engineering Priorities	13
12	Reproducibility	13
12.1	Test suite	13
12.2	V2.1 immediate-boundary model	13
12.3	Selective-inclusion counterfactual	13
12.4	Live relay query	14
12.5	Generated evidence	14
13	Companion Documents	14
14	References	14

1 Executive Summary

GridPool is an experimental Bitcoin reward-sharing protocol. Miners construct their own block templates, retain the block-finder output and transaction fees, and commit a common payout suffix directly in the coinbase. There is no pool wallet, later payout promise, or separate sharechain.

The core innovation is **ranked achieved difficulty**. Conventional pool ledgers normally count shares at an assigned target difficulty. GridPool instead keeps a bounded set of the best actual proof-of-work outcomes. The cutoff of a fixed-depth list is an order statistic that estimates the accumulated work that produced it. This allows sparse, independently verifiable accounting without retaining every low-difficulty share.

V2.1 separates:

- an Unpaid Work Set of up to 897 validated proofs;
- an Active Snapshot containing the fixed post-slot-0 payout suffix; and
- Paid Lineage identifying exactly which proof IDs a GridPool block consumed.

Every Bitcoin block snapshots the best unpaid work without clearing it. A real GridPool block pays the active snapshot and removes only the proof IDs actually paid. Under the intended V2.1 rule, compatible proofs on the current Bitcoin parent merge forward only within the same active payout state, while unknown previous-parent proofs arriving after a node's local boundary cannot rewrite it. Candidate-state IDs are anchored to the active snapshot. A new runtime regression confirms that a proof carried by a genuinely different active payout snapshot cannot be imported into the local candidate state.

The evidence is promising but bounded:

- modeled 300-slot payouts substantially reduce actual BTC variance in mature team scenarios;
- proof claims cannot be forged through pool hopping or Sybil identities;
- straightforward profit-seeking block withholding is costly in the current model;
- compact relay sharply reduces immediate boundary disagreement in a stylized 24-node model; and
- a deliberately permissive selective-inclusion model found a large free-riding transfer if proofs from genuinely different active payout states were cross-credited, while the runtime regression showed that the reference implementation rejects that transition; and
- preliminary public-node telemetry confirms that UDP, WebSocket, and HTTP arrivals are observable and that UDP is often first.

This handout does not claim solved Byzantine consensus. The implementation audit identified direct-ingress stale-parent admission, cross-snapshot paid lineage, fee-variant lineage, and recovery from a genuine active-snapshot split as cases that need explicit fixes, policy, or tests before V2.1 should be considered consensus-complete.

2 What Is New In This Revision

This document preserves the original July 17 handout separately and corrects five points:

1. The Active Snapshot is the common **post-slot-0 suffix**, not slot 0 plus the suffix.
2. An ordinary Bitcoin block snapshots unpaid work but removes nothing; a GridPool block first records its proof, then removes the IDs it paid.
3. A latency split does not merge a late old-parent proof after the boundary. Only new current-parent work merges forward.
4. The existing latency simulation measures immediate boundary disagreement, not complete recovery duration. A strong boundary-only proof can keep two payout snapshots different beyond the next Bitcoin block.
5. Merge-forward is not cross-branch credit. Candidate states are derived from the active state, so genuinely different active payout snapshots form separate candidate-state domains in the tested runtime path.

3 The Ranked-Proof Idea

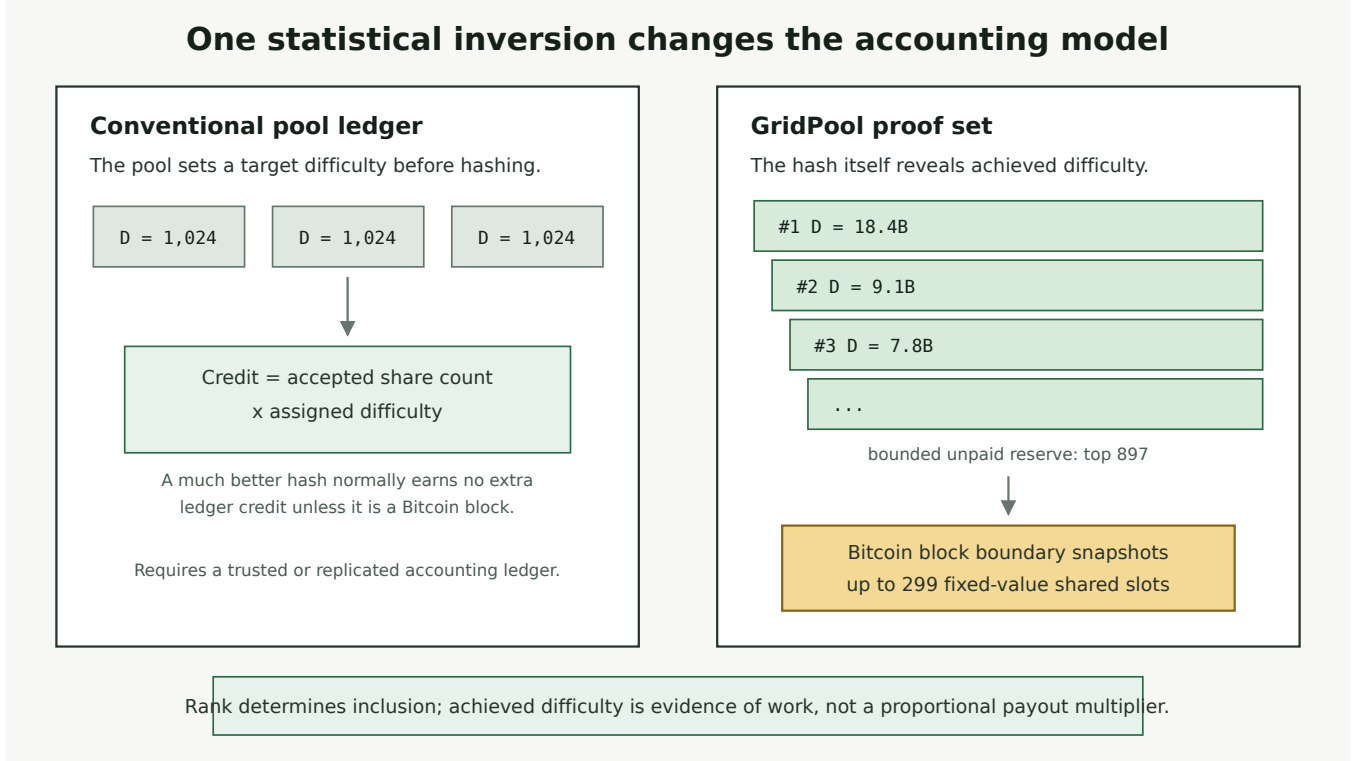


Figure 1: Conventional and ranked-proof accounting

For an admission difficulty $d_{\min}$, a qualifying hash's achieved difficulty D satisfies

$$\Pr[D \geq x \mid D \geq d_{\min}] = \frac{d_{\min}}{x}.$$

Therefore $V=d_{\min}/D$ is approximately uniform on $(0,1)$. If S qualifying hashes were generated and $D_{(m)}$ is the m -th best achieved difficulty, then

$$\hat{S} \approx m \frac{D_{(m)}}{d_{\min}}.$$

An unbiased fixed- S form is $(m-1)/V_{(m)}$, with relative standard error approaching $1/\sqrt{m-2}$. That is approximately:

Retained rank	Approximate relative standard error
300	5.79%
897	3.34%

The distribution is heavy-tailed, so summing achieved difficulties is dominated by rare outliers. The fixed-rank cutoff is a more stable estimate of accumulated work for full equal-depth lists.

This is a ranking mechanism, not proportional pay-per-difficulty. A selected proof earns one fixed shared slot. Higher achieved difficulty improves its rank and survival probability.

4 V2.1 State and Payout Construction

Object	Meaning
Unpaid Work Set	Top 897 distinct, validated, unpaid proof IDs, sorted by achieved difficulty.
Active Snapshot	Common post-slot-0 payout suffix frozen at a Bitcoin block boundary.
Snapshot Context	Payout suffix against which a proof's coinbase was mined.
Paid Lineage	Exact proof IDs consumed by an accepted GridPool block.

The default coinbase has 300 conceptual positions. Let

$$q = \left\lfloor \frac{\text{subsidy}}{300} \right\rfloor.$$

Slot 0 receives one conceptual slot, transaction fees, integer remainder, and any unfilled bootstrap value. The common suffix is either:

- canonical support output plus up to 298 shared proof positions; or
- support disabled plus up to 299 shared proof positions.

The support destination is canonical when enabled; an operator cannot replace it with a custom fee address. Duplicate payout scripts may be condensed into a smaller number of transaction outputs while preserving value.

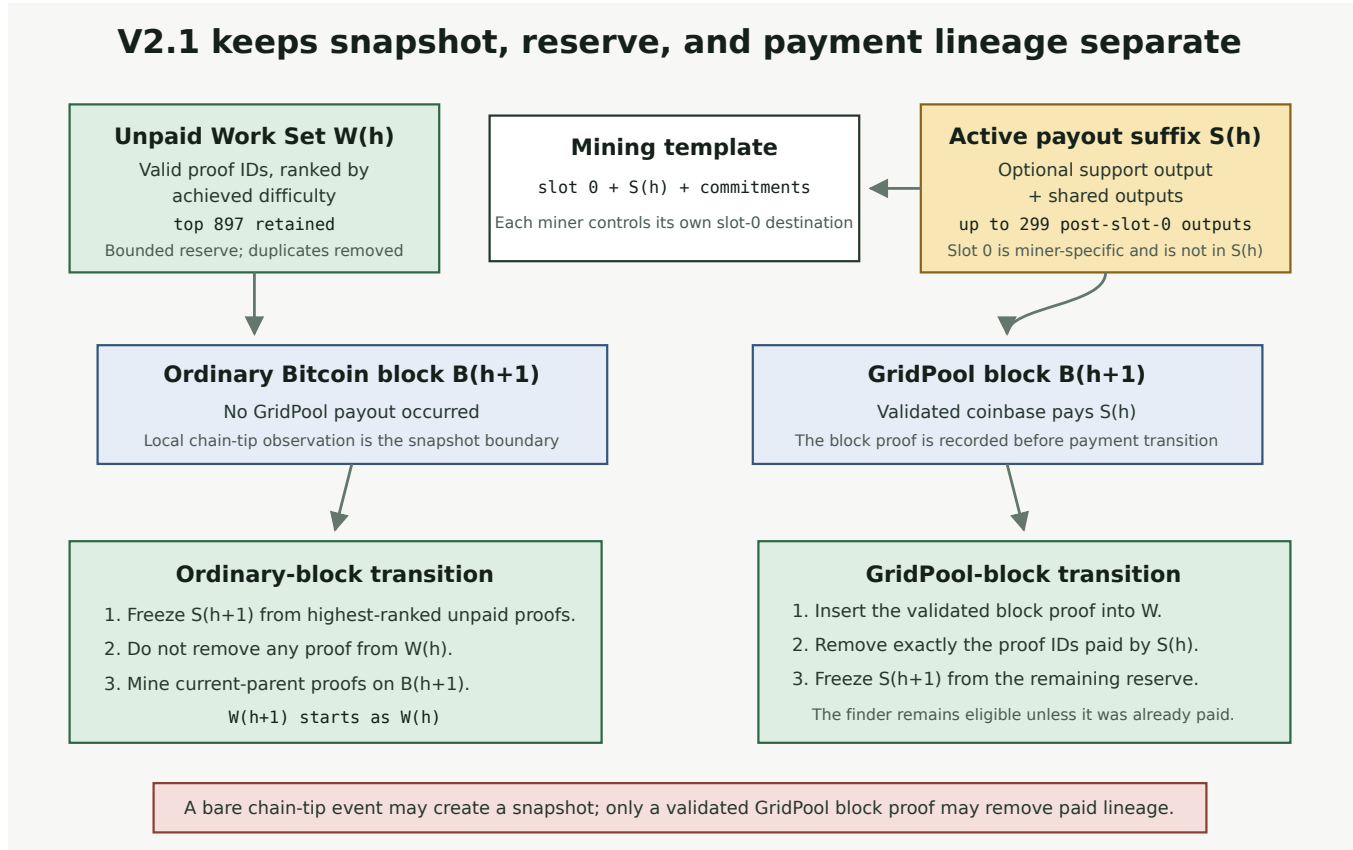


Figure 2: V2.1 state lifecycle

4.1 Ordinary Bitcoin block

1. Freeze the next Active Snapshot from the top unpaid proofs.
2. Preserve the entire Work Set because no GridPool payout occurred.
3. Move mining to the new Bitcoin parent.

4.2 GridPool block

1. Validate and insert the block proof into the Work Set.
2. Identify the snapshot actually paid by the block's coinbase.
3. Remove exactly those paid proof IDs once.
4. Build the next Active Snapshot from the remaining reserve.

The finder therefore earns slot 0 and fees immediately and normally remains eligible for a future shared position. A chain-tip notification can create an ordinary snapshot, but it must not remove lineage without a validated GridPool block proof.

5 Boundary Races and Merge-Forward Recovery

If two peers share the same active payout state but have different valid proofs on the **current** Bitcoin parent, the correct operation is union, deduplication, deterministic sort, and trim to 897. It is not a vote between whole candidate lists.

That rule does not authorize cross-credit between genuinely different active payout snapshots. Candidate-state IDs commit to the active state as well as the candidate Work Set. A bundle built on active snapshot **S_B** therefore does not recompute to the same candidate ID under a node whose active snapshot is **S_A**. The tested import path rejects it rather than silently crediting the proof on both branches.

Nodes cannot trust peer clocks. Under the V2.1 rule, once a node locally observes the next Bitcoin block, an unknown proof on the previous parent cannot be used to rewrite its Active Snapshot or canonical future reserve. This prevents an attacker from backdating intentionally stale work and invoking a deterministic “heavier stale list wins” rule.

The candidate-bundle import path implements this rule. The direct share-ingress path does not yet enforce it consistently: its validation parent set retains the immediate previous parent and parents represented in the reserve. This is a P0 implementation gap, not a change to the intended V2.1 rule.

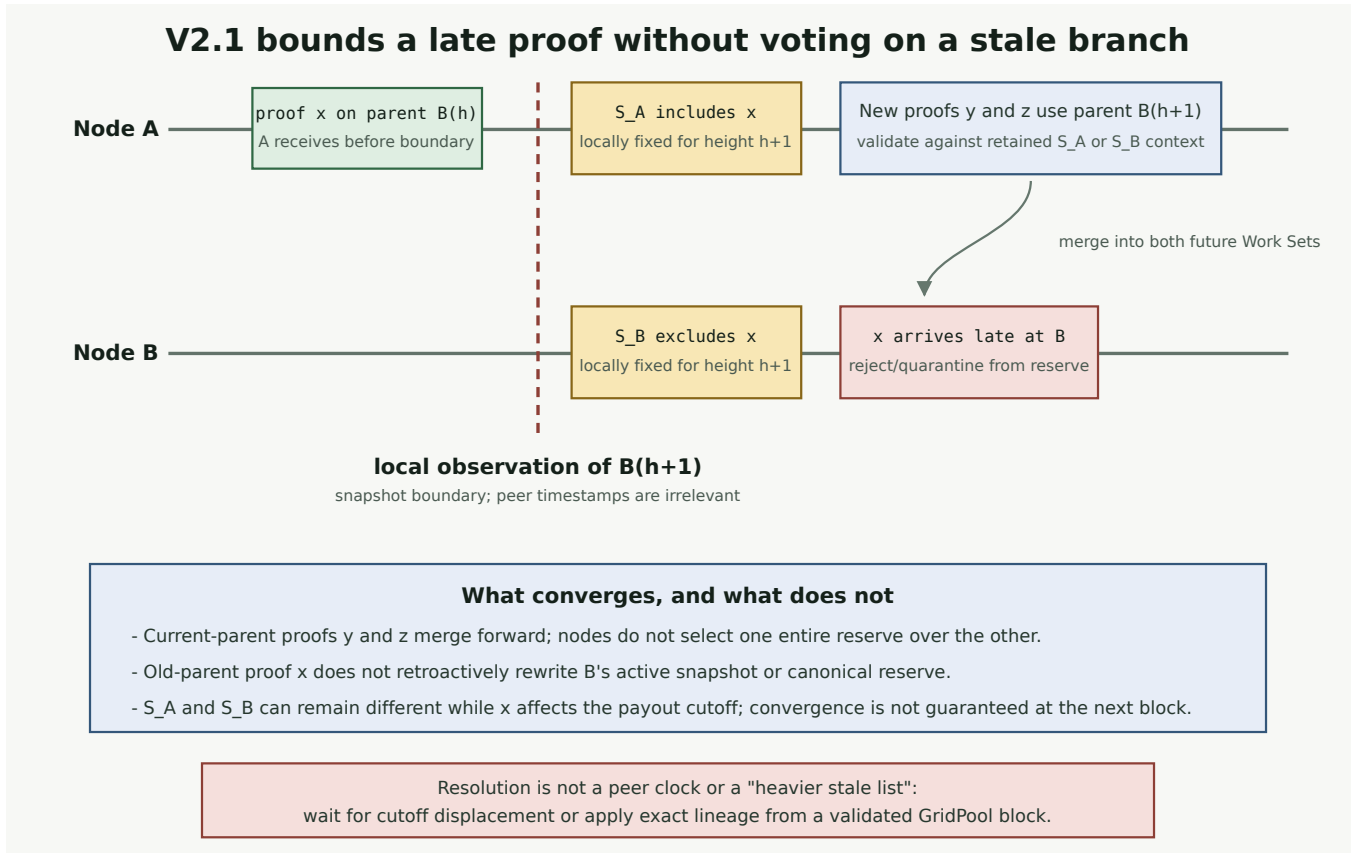


Figure 3: Corrected V2.1 boundary timeline

A proof seen by A before the boundary but by B after it can create $S_A \neq S_B$. B rejects or quarantines that old-parent proof. New proofs on the current parent can merge when their retained context belongs to the same active payout state. A genuinely different active payout state remains a separate branch unless an explicit, independently validated recovery rule is added.

This is narrower than a sharechain fork, but it is not guaranteed one-block convergence. If the missed proof remains payout-cutoff-relevant, the active suffixes can remain different until:

- the proof is displaced from the relevant cutoff; or
- a valid GridPool block identifies the paid snapshot and every node applies exactly that snapshot's lineage.

That second path is consensus-critical and is not yet fully regression-covered for cross-snapshot blocks. Automatic recovery from a genuine active-snapshot split also remains an open policy problem; the current runtime correctly avoids solving it by cross-crediting arbitrary divergent work.

6 Findings

6.1 Actual BTC payout variance

The variance study models **actual BTC paid**, not merely probability of holding a slot. The standard scenario covers 30 days, 4,320 expected Bitcoin blocks, a 600 EH/s reference network, 3.125 BTC subsidy, and 0.05 BTC fees.

Variance Reduction Versus Solo By Slot Count

Mean across miner sizes after applying the same filter as the CV chart.

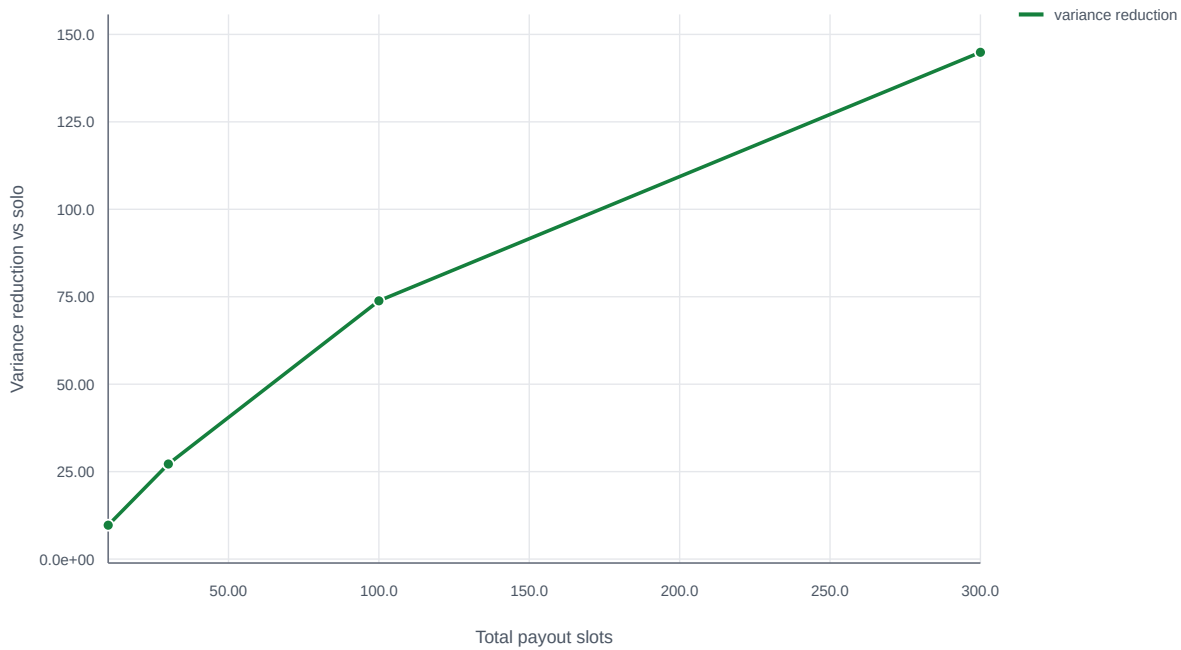


Figure 4: Variance reduction by payout slots

The mature-team asymptote for 300 total slots is about 271.7x variance reduction versus solo in this model, not exactly 300x. The same variance-reduction curve applies across miner sizes when team size is expressed as a multiple of miner hashrate. Absolute payout cadence still depends on the miner's share of Bitcoin network hashrate and the team's block cadence.

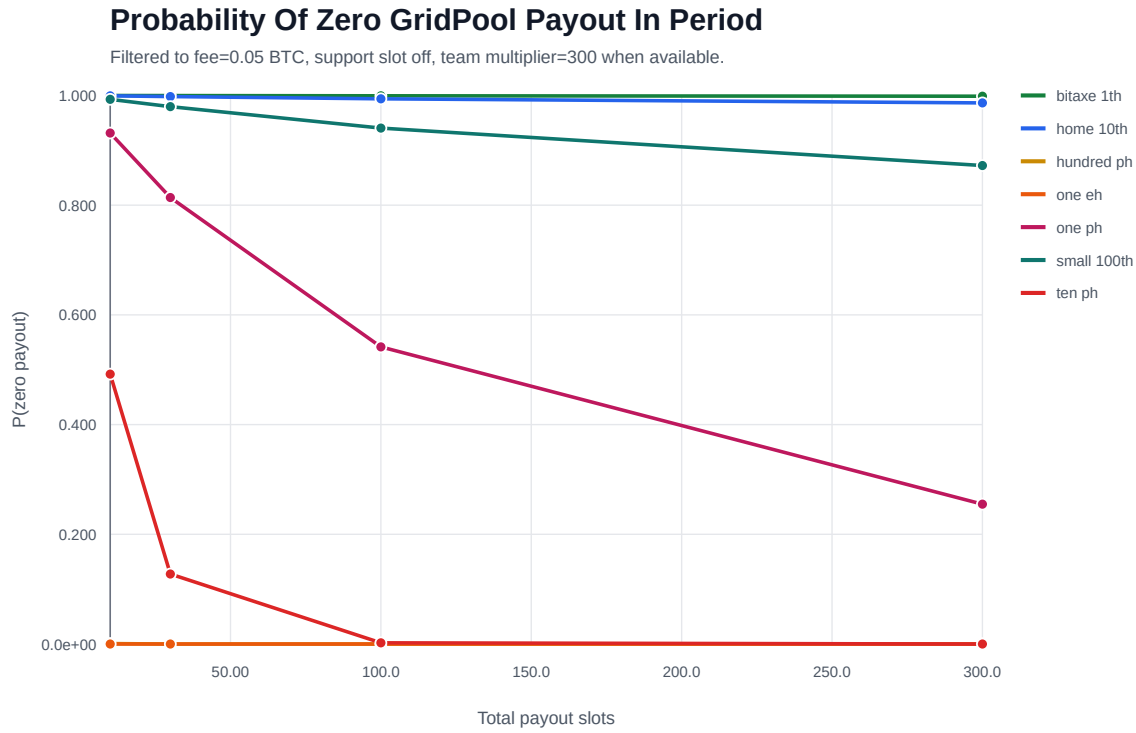


Figure 5: Zero-payout probability by payout slots

The important limitation is bootstrap scale. A broad payout list cannot create blocks. If the team is too small to find blocks at a useful cadence, a tiny miner remains lottery-like even when expected value is fair.

6.2 Pool hopping

An earned GridPool proof is a claim backed by real work. Leaving cannot forge a claim, alter slot-0 attribution, or erase another miner's lineage. The departing miner gives up future shared-proof chances, slot 0, fees, and acceleration of its existing claims.

The models do show a limited free-option effect when the outside alternative is idealized as zero-fee and deterministic or when a large miner can tolerate solo variance. The measured strategy effects are small and condition-dependent. The defensible conclusion is:

Hopping may change how a miner allocates new work, but it cannot create fake past work or debit honest miners' proof claims.

6.3 Block withholding

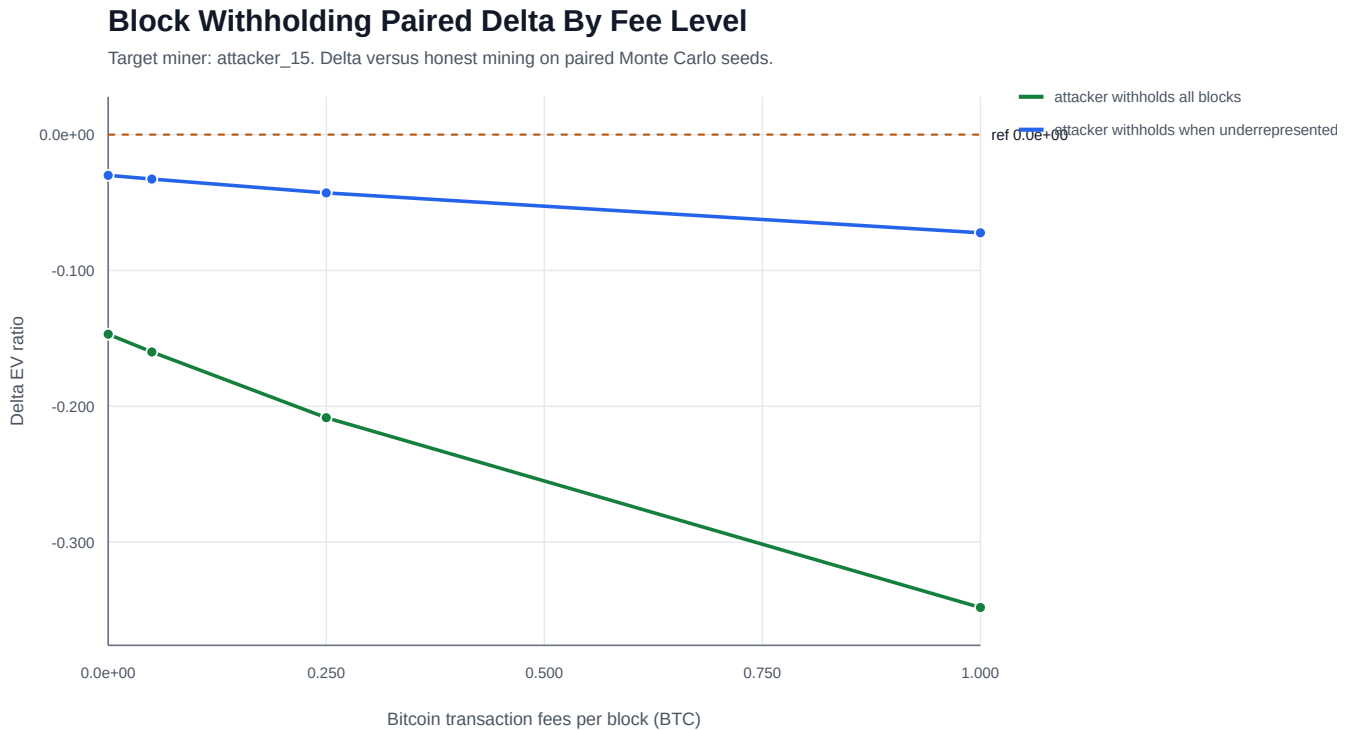


Figure 6: Attacker outcome by transaction-fee level

Withholding a valid GridPool block sacrifices slot 0, transaction fees, and faster payment of the attacker’s own shared claims. Straightforward profit-maximizing withholding underperforms honest mining in the modeled cases. This does not rule out an attacker willing to spend money for disruption.

6.4 Selective inclusion and active-state anchoring

A new stress test asked whether a miner could exclude everyone else’s proofs from its own payout snapshot while still relaying its current-parent proofs to an inclusive team. The model deliberately allowed those proofs to be credited across genuinely different active payout states. Each row below summarizes 20 replications of 3,000 Bitcoin blocks at 3% GridPool network share.

Attacker hash share	Modeled reward share under cross-state credit	Excess reward share	95% interval for excess
10%	16.775%	+6.775 pp	+5.403 to +8.147 pp
35%	47.921%	+12.921 pp	+11.906 to +13.936 pp
51%	64.518%	+13.518 pp	+12.447 to +14.590 pp
67%	77.989%	+10.989 pp	+9.860 to +12.118 pp
90%	94.357%	+4.357 pp	+3.779 to +4.935 pp

The equal and opposite honest-miner loss, together with payout-conservation error below $6.25\text{e-}13$ BTC, shows a transfer rather than value creation. The effect is therefore a strong warning: **a future recovery rule must not blindly credit proofs across different active payout states.**

This is a counterfactual result, not a confirmed runtime exploit. A regression constructed two genuinely different active payout snapshots, built a valid current-parent proof on the exclusionary one, and attempted candidate-bundle import into the inclusive node. Import was rejected and the inclusive Work Set remained unchanged. Candidate IDs

are anchored to the active state, so the permissive transition assumed by the stress model is not reachable through the tested V2.1 candidate-import path. Private-split and canonical-context-only controls remained near fair expected value within simulation noise.

The resulting engineering conclusion is narrower and stronger than either model alone: same-active-state evidence should merge, different-active-state evidence must not be cross-credited by default, and genuine split recovery needs an explicit lineage-preserving rule rather than a generic “heavier state wins” or “merge everything current-parent” shortcut.

6.5 Immediate boundary disagreement by relay profile

The V2.1 boundary model uses 24 nodes, target degree 8, 299 shared slots, an 897-proof reserve, 220 relay-worthy shares per Bitcoin interval, and 60,000 simulated boundaries per relay profile (5,000 x 12).

Relay profile	Boundaries with more than one snapshot	Mean missing shared positions per node	Shared position/node loss
JSON HTTP	61.0283%	0.2375	0.079442%
Compact WebSocket	11.6200%	0.0138	0.004606%
UDP fast path with fallback	0.0550%	<0.0001	0.000009%

The high raw split rate for JSON should be read alongside the much smaller position-loss rate: many modeled splits differ by only one position at one or a few nodes.

The model’s payload totals are aggregate modeled delivery bytes across all 24 nodes and the full 60,000-boundary run, not bytes per node or per snapshot:

Relay profile	Full-run modeled payload	Equivalent modeled payload/day across all 24 destinations
JSON HTTP	4,536.68 MB	about 10.89 MB/day
Compact WebSocket	1,852.55 MB	about 4.45 MB/day
UDP with fallback	1,850.82 MB	about 4.44 MB/day

The daily conversion uses 60,000 x 600 seconds = 416.7 days of simulated Bitcoin time. It is a payload approximation, not measured Internet billing and not edge-hop-complete network traffic.

Most importantly, this model measures **boundary inclusion**, not how long a cutoff-relevant disagreement persists. Its original “recovery” name is broader than the behavior actually simulated.

7 Preliminary Live Relay Telemetry

The following bounded query was collected on 2026-07-13 from a 12-hour API window. It is a field sanity check, not statistical proof. Evomining was not reachable during this specific query, so the table includes only Main and Dallas.

7.1 Pulse-proof arrivals

Receiving node	Transport	Arrivals	First arrivals	Accepted	Duplicates	Median delta from first	P95 delta	Average payload
Main	UDP	575	575	564	1	0 ms	0 ms	877 B
Main	Web-Socket	701	123	122	536	16.31 ms	244.23 ms	4,355 B
Main	HTTP JSON	701	11	11	141	23.08 ms	250.26 ms	2,697 B

Receiv- ing node	Trans- port	Arrivals	First arrivals	Accepted	Duplicates	Median delta from first	P95 delta	Average payload
Dallas	UDP	1,409	1,409	1,324	69	0 ms	0 ms	874 B
Dallas	Web- Socket	1,409	60	61	1,024	8.54 ms	21.84 ms	4,382 B
Dallas	HTTP JSON	1,504	22	23	309	13.40 ms	170.49 ms	unavail- able

Delta from first is receiver-local: the first observed transport for a proof is zero, and later copies are measured against it. Counts are not a controlled pairwise benchmark because routes, peers, missing copies, and duplicates differ.

The safe interpretation is:

The public beta can observe redundant transport arrival, UDP frequently supplies the first copy, and compact UDP payloads are roughly one fifth of the WebSocket payload in this sample. The topology and sample are too small to infer global latency or a production split-rate reduction.

Work-set-mutating proofs were sparse in this window (18 UDP arrivals at each queried node), which is why pulse proofs are useful for ongoing measurement.

8 Security Claims and Limits

Topic	What V2.1 provides	What it does not provide
Sybil identities	Addresses and nodes create no proof-of-work claims.	Connection and eclipse resistance without network controls.
Share forgery	Header, coinbase, Merkle path, parent, snapshot, and slot 0 are independently verified.	Protection from every parser or resource-exhaustion bug.
Pool hopping	Past claims remain proof-bound and paid once.	A promise that every outside-market condition makes leaving irrational.
Block withholding	Finder sacrifices slot 0, fees, and claim acceleration.	Protection from externally motivated griefing.
Stale rewrite	Unknown old-parent proofs cannot rewrite a local boundary.	Perfect agreement among nodes that observed events in different orders.
Selective inclusion	Candidate IDs bind candidate work to the active payout state; cross-state import is rejected in the tested path.	Automatic reconciliation of genuinely different active snapshots.
Transaction censorship	Pool relay does not carry the full transaction list; miners build templates locally.	Cryptographic anonymity or invisibility once a block is public.
Sharechain attack	There is no secondary chain to reorg.	Immunity to majority censorship, partition, or team splitting.

9 Regression and Implementation Audit

The full reference suite passed on 2026-07-14; the source currently contains 102 test methods.

Invariant	Why it matters	Test status	Source / next step
Ordinary snapshots preserve unpaid reserve	Bitcoin blocks must not erase unpaid work.	Covered	<code>ShareAttributionTests.cs:1085</code>
Compatible current-parent work merges within one active state	Honest peers should union evidence, not elect whole lists.	Covered	<code>CandidateImportMergesCurrentParent</code>
Different active payout states cannot cross-credit candidate proofs	Prevents exclusionary branches from free-riding on an inclusive team.	Covered for candidate import	<code>CandidateImportRejectsCurrentParent</code>
Unknown old-parent work cannot cross local boundary	Blocks backdated stale-branch replacement.	Candidate import covered; direct ingress not enforced	import test at :974; add ingress race tests
Retained contexts validate old unpaid proofs	Snapshot churn must not invalidate honest reserve work.	Covered	context tests at :1361-1477
Local active payment removes only paid IDs	Preserves reserve and enables lucky-streak depth.	Covered in ordinary local path	payment tests at :1139-1209
Duplicate block does not pay twice	Paid-once transition must be idempotent.	Covered	duplicate test at :832
Support fee on/off constructs valid suffixes	Optional canonical support must not alter slot count.	Partially covered	construction covered; cross-variant payment lineage missing
State bundle cannot smuggle stale proofs	Recovery must not bypass boundary finality.	Partially covered	stale-parent import test exists; add adversarial bundle matrix
Block paying retained divergent snapshot removes its actual IDs	Required to reconcile a real block across a latency split.	Missing; likely implementation gap	pass paid snapshot ID through payment transition and test both receivers
Fee-free block removes all fee-free paid proof IDs	Prevents one paid proof remaining eligible on fee-enabled peer.	Missing; likely lineage gap	retain proof IDs per payout variant and add cross-config test
Recovery fetches lower-total unique current proof	Merge semantics should not depend on aggregate score.	Missing	make recovery set-difference-aware rather than total-difficulty-gated
Genuine active-snapshot split recovery	Honest boundary races need a bounded way to heal without enabling cross-state free riding.	Policy and tests missing	specify lineage/overlap constraints; do not use unconditional cross-state merge

10 What The Evidence Supports

The July 17 presentation can defensibly say:

- GridPool’s statistical primitive is a well-defined order statistic, not a heuristic share score.
- Fixed-value broad snapshots can reduce actual BTC payout variance while preserving expected attribution.
- V2.1’s merge-forward design removes routine whole-list “heaviest branch” selection for compatible current-parent work within the same active state.
- Candidate-state anchoring blocks the tested cross-active-state selective- inclusion path. The simulation shows why weakening that boundary would be dangerous, not that the current runtime is exploitable through it.
- Local non-retroactive boundaries can close the simple backdated stale-work entry path without trusting peer clocks once all ingress paths enforce them.

- Fast relay materially reduces immediate boundary disagreement in the model, and live telemetry confirms that the transports are measurable.
- The reference implementation has substantial regression coverage and a small, explicit list of consensus-critical gaps.

It should not say:

- all nodes always converge at the next Bitcoin block;
- GridPool is immune to 51% miners, partitions, or censorship;
- UDP has been proven globally superior from a two-operator sample;
- all miner firmware supports a 300-address coinbase; or
- the current implementation is a finished interoperability specification.

11 Immediate Engineering Priorities

1. Enforce the local boundary on direct share ingress. Permit old parents for validation of already-known lineage, not admission of a newly received proof.
2. Bind GridPool payment removal to the snapshot proven by the winning block, not merely the receiver's current active snapshot.
3. Store paid proof lineage separately for support-on and support-off variants, or remove cross-variant consensus until it can be unambiguous.
4. Add direct-ingress, cross-snapshot, and cross-config payment tests.
5. Model disagreement persistence through cutoff displacement and actual GridPool payment, extending the immediate-boundary model.
6. Continue multi-node UDP/pulse telemetry with explicit run windows, restarts, topology, and paired-proof counts.
7. Publish a normative V2.1 specification and implementation-independent test vectors.
8. Specify bounded recovery for genuine active-snapshot splits without allowing unconditional cross-state proof credit. Treat the selective-inclusion model as a regression oracle for proposed recovery rules.

12 Reproducibility

12.1 Test suite

```
cd /home/keegreil/Documents/GitHub/boot-protocol
dotnet test boot.tests/boot.tests.csproj --no-restore --verbosity minimal
```

12.2 V2.1 immediate-boundary model

```
cd /home/keegreil/Documents/GitHub/gridpool-simulations
python3 run_v21_latency_recovery.py \
  --out-dir reports/generated/v21_latency_recovery_july17_long \
  --blocks 5000 \
  --replications 12 \
  --node-count 24 \
  --peer-degree 8 \
  --shared-slots 299 \
  --reserve-multiplier 3 \
  --shares-per-block 220
```

12.3 Selective-inclusion counterfactual

```
cd /home/keegreil/Documents/GitHub/gridpool-simulations
python3 run_v21_selective_inclusion.py \
  --out-dir reports/generated/v21_selective_inclusion_long \
  --blocks 3000 \
  --replications 20
```

The permissive `merge_all_current_parent` policy is intentionally broader than the tested runtime. Compare it with `canonical_context_only`, then run the `CandidateImportRejectsCurrentParentProofFromDifferentActiveSnapshotAsync` regression in `boot-protocol` to check reachability.

12.4 Live relay query

```
cd /home/keegreil/Documents/GitHub/boot-protocol
node scripts/peer-relay-latency-report.mjs \
  --url https://main.gridpool.net \
  --window 12h \
  --limit 5000
```

12.5 Generated evidence

Topic	Path
Payout variance	reports/generated/sweeps/payout_variance_fee_long/
Pool hopping	reports/generated/sweeps/pool_hopping_external_mode_299_1/
Block withholding	reports/generated/sweeps/block_withholding_fee_sensitivity/
V2.1 boundary inclusion	reports/generated/v21_latency_recovery_july17_long/
Selective-inclusion counterfactual	reports/generated/v21_selective_inclusion_long/
Live telemetry	reports/july17/live-telemetry/

13 Companion Documents

- Draft whitepaper: `gridpool-spec/gridpool-whitepaper-draft.md`
- Reference implementation: `boot-protocol`
- Modeling code and generated evidence: `gridpool-simulations`
- V2.1 implementation audit: `boot-protocol/docs/consensus-selection-audit.md`
- Critic FAQ: `boot-protocol/docs/critic-faq.md`

14 References

1. Satoshi Nakamoto, Bitcoin: A Peer-to-Peer Electronic Cash System.
2. Meni Rosenfeld, Analysis of Bitcoin Pooled Mining Reward Systems.
3. BIP 22: `getblocktemplate` - Fundamentals.
4. Stratum V2 Template Distribution Protocol.
5. GridPool source, tests, scenarios, and generated reports cited above.